

Flight Delay Analysis

Problem Statement:

An airline company maintains flight records including departure/arrival times and delays. The dataset contains delays caused by weather, airline operations, or technical issues. The HR Analytics team needs to analyze delays by filtering and grouping operations using Pandas (no visualization).

Name:

Chandana K T (4GW23CI009)

Email:

chandanakt1712@gmail.com

Date:

03-09-2025

INDEX

- Project Description
- System Requirements
- Architecture
- Use Cases
- Data Model
- Algorithms
- Flowchart
- Project structure
- Front-end (Interface) Design
- Code Snippet
- Output
- Web-page
- Testing & Results
- Conclusion
- Future Work

Project Description

The **Flight Delay Analysis System** is a Flask-based web application designed to study and analyze flight delay records using pre-collected airline datasets. It enables users to filter flights based on delay duration, examine airline performance summaries, view airport-level statistics, and explore route-based delay trends.

The backend, built with **Python and Pandas**, handles efficient data processing, while the frontend, developed with **HTML, CSS, Bootstrap, and Jinja2 templates**, provides a clean and user-friendly interface. This system helps students, researchers, and aviation stakeholders gain insights into flight punctuality and operational challenges, while also laying the foundation for future enhancements such as real-time data integration and predictive analysis.

System Requirements

Functional Requirements:

- User can filter flights with delays above a threshold.
- View airline summary (on-time vs delayed).
- View airport summary (departures, arrivals, delay rates).
- Explore route summaries and performance trends.
- Display results in tabular and graphical formats.

Non-Functional Requirements:

- Fast response time for data queries.
- Cross-browser compatibility.

- Responsive web design.
- Scalability to handle larger datasets.

Architecture

The project follows a client-server architecture. The front-end (HTML, CSS, JS) communicates with Flask (Python) backend, which processes data using Pandas and serves results.

Static files such as JavaScript handle dynamic updates, while Jinja2 templates render views.

The architecture includes the following layers:

- Presentation Layer: HTML, CSS, JS
- Application Layer: Flask routes (app.py)
- Data Layer: CSV datasets (flights, airlines, airports, routes)

Use cases

Use Case 1: Filter delayed flights

Actors: User

Preconditions: Data loaded

Main Flow:

1. User selects delay filter (e.g., > 30 min)
2. Flask queries dataset
3. Results shown in table

Use Case 2: View airline performance

Actors: User

Main Flow:

1. User clicks airline summary
2. System displays performance chart

Data model

Datasets:

- flights.csv: Flight-level data with departure delays.
- airline_summary.csv: Aggregated airline performance.
- airport_summary.csv: Airport-level arrivals/departures.
- route_summary.csv: Route performance statistics.

Relationships:

- Airlines operate flights.
- Flights connect airports.
- Routes are combinations of origin and destination airports.

Algorithms

The project primarily uses Pandas-based filtering and aggregation algorithms. Example:

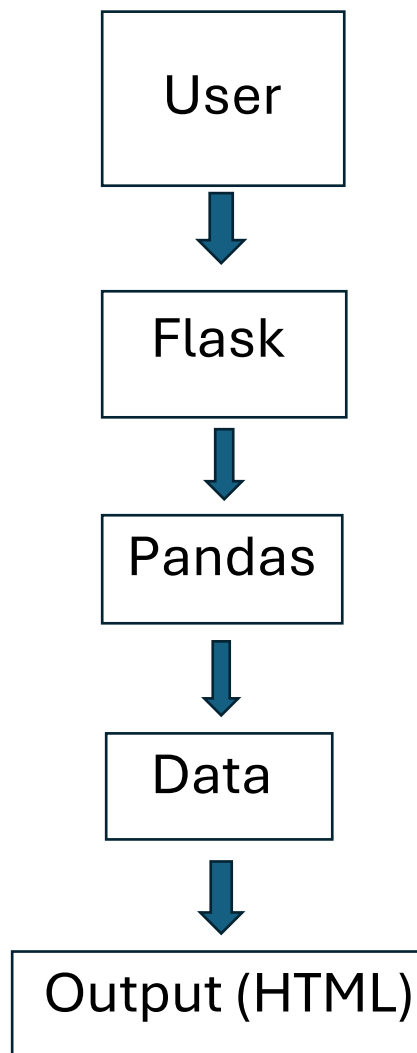
Eligibility for 'delayed30' filter:

Algorithm:

1. Load flights.csv
2. Apply condition DepartureDelay > 30
3. Return filtered dataset

This simple linear scan operates in $O(n)$ time where n is number of records.

Flowchart



Project Structure (with frontend)

airline-delays/

├─ app.py # Flask backend

├─ flights.csv # dataset

├─ requirements.txt

├─ templates/

| └─ index.html # frontend

```
└─ static/  
  └─ main.js      # frontend logic
```

Frontend (interface) Design

The interface is rendered using Jinja2 templates with Bootstrap for styling.

Key pages:

- Home Dashboard (index.html)
- Delayed Flights Filter
- Airline Summary View
- Airport Summary View

The design emphasizes clean navigation and responsive layout

Code Snippet

analysis.py

```
import pandas as pd  
  
# 1. Load dataset  
df = pd.read_csv("data/flights.csv")  
  
# Clean up column names (remove spaces, standardize)  
df.columns = df.columns.str.strip()  
  
# 2. Explore dataset  
print("---- HEAD ----")  
print(df.head())  
print("\n---- INFO ----")  
print(df.info())  
print("\n---- DESCRIBE ----")  
print(df.describe(include="all"))
```

The provided Python code uses the pandas library to perform basic data exploration on a dataset named 'flights.csv'. It loads the data into a

DataFrame, standardizes the column names by removing extra spaces, and then prints the first few rows using `.head()` and a summary of the data types and non-null values using `.info()`. Finally, it generates descriptive statistics for all columns, including text-based and numerical data, using `.describe(include="all")`

```
# 3. Filtering
print("\n---- FILTERING ----")

# Flights delayed more than 30 mins
delayed_30 = df[df["DepartureDelay"] > 30]
print("Flights delayed > 30 mins:\n", delayed_30)

# Flights from JFK
jfk_flights = df[df["OriginAirport"] == "JFK"]
print("\nFlights from JFK:\n", jfk_flights)

# Flights by Delta
delta_flights = df[df["Airline"] == "Delta"]
print("\nFlights by Delta:\n", delta_flights)

# 4. Grouping
print("\n---- GROUPING ----")
```

The Python code uses **boolean indexing** to filter a pandas DataFrame based on specific conditions. It creates three new DataFrames: one containing flights with a departure delay of more than 30 minutes, another with flights originating from "JFK," and a third with flights operated "Delta." Each resulting DataFrame is then printed to display the filtered results.


```

# By Airline → average delay, total flights
airline_summary = df.groupby("Airline").agg(
    avg_departure_delay=("DepartureDelay", "mean"),
    total_flights=("FlightID", "count")
).reset_index()
print("\nAirline Summary:\n", airline_summary)

# By OriginAirport → total flights, % delayed
airport_summary = df.groupby("OriginAirport").agg(
    total_flights=("FlightID", "count"),
    delayed_flights=("DepartureDelay", lambda x: (x > 0).sum())
)
airport_summary["percent_delayed"] = (
    airport_summary["delayed_flights"] / airport_summary["total_flights"] * 100
)
airport_summary = airport_summary.reset_index()
print("\nAirport Summary:\n", airport_summary)

# By Date → daily average delay
date_summary = df.groupby("Date").agg(
    daily_avg_delay=("DepartureDelay", "mean")
).reset_index()
print("\nDate Summary:\n", date_summary)

# By Origin-Destination route → avg delay
df["Route"] = df["OriginAirport"] + "-" + df["DestinationAirport"]
route_summary = df.groupby("Route").agg(
    avg_delay=("DepartureDelay", "mean"),
    total_flights=("FlightID", "count")
).reset_index()
print("\nRoute Summary:\n", route_summary)

```

This Python code uses the **pandas** `groupby()` method to perform data aggregation. It creates summaries by calculating average delays and total flight counts for various categories like airlines, airports, dates, and flight routes.

```

# 5. Export summaries
airline_summary.to_csv("airline_summary.csv", index=False)
airport_summary.to_csv("airport_summary.csv", index=False)
route_summary.to_csv("route_summary.csv", index=False)

print("\n✅ Summaries exported: airline_summary.csv, airport_summary.csv, route_summary.csv")

```

The provided Python code saves three previously created summary DataFrames—`airline_summary`, `airport_summary`, and `route_summary`—to separate CSV files. The `.to_csv()` method is used with the `index=False` parameter to ensure the output files do not include the DataFrame's index column. A final message confirms the successful export of the files.

app.py

```
from flask import Flask, render_template
import pandas as pd

app = Flask(__name__)

# Load dataset
df = pd.read_csv("data/flights.csv")
```

The Flask application loads flight data from a CSV file and uses pandas to process it. It then renders a web page with the data presented in a clean, responsive HTML table using the Tailwind CSS framework.

```
@app.route("/")
def home():
    return render_template("index.html", title="Dashboard", table=None)

@app.route("/filter/delayed30")
def delayed30():
    result = df[df["DepartureDelay"] > 30]
    return render_template("index.html",
                           title="Flights Delayed > 30 mins",
                           table=result.to_html(classes="table table-bordered table-striped", index=False))

@app.route("/filter/jfk")
def jfk():
    result = df[df["OriginAirport"] == "JFK"]
    return render_template("index.html",
                           title="Flights from JFK",
                           table=result.to_html(classes="table table-bordered table-striped", index=False))

@app.route("/filter/delta")
def delta():
    result = df[df["Airline"] == "Delta"]
    return render_template("index.html",
                           title="Flights by Delta",
                           table=result.to_html(classes="table table-bordered table-striped", index=False))
```

This Python code extends the Flask application to include new routes for filtering the flight data. Each new route (/filter/delayed30, /filter/jfk, and /filter/delta) performs a specific filtering operation on the DataFrame. It uses the to_html() method to convert the filtered results into an HTML table and passes this table to the index.html template. This allows the web application to display different subsets of the data based on the URL requested by the user.

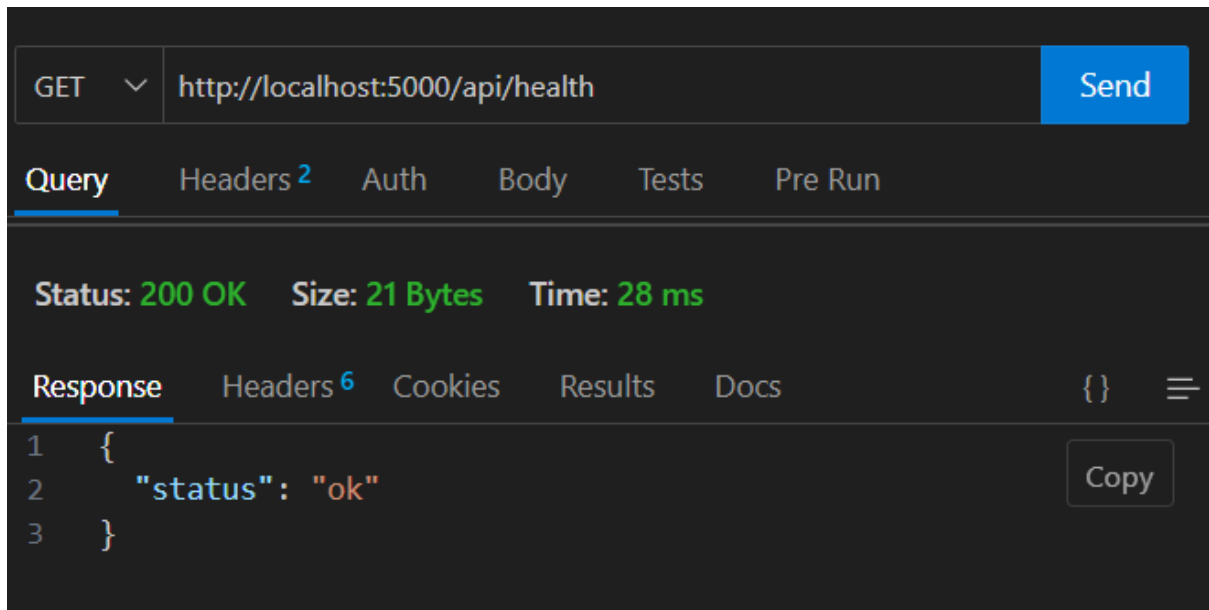
```
@app.route("/summary/airlines")
def summary_airlines():
    result = df.groupby("Airline").agg(
        avg_departure_delay=("DepartureDelay", "mean"),
        total_flights=("FlightID", "count")
    ).reset_index()
    return render_template("index.html",
                           title="Airline Summary",
                           table=result.to_html(classes="table table-bordered table-striped", index=False))

@app.route("/summary/airports")
def summary_airports():
    result = df.groupby("OriginAirport").agg(
        total_flights=("FlightID", "count"),
        delayed_flights=("DepartureDelay", lambda x: (x > 0).sum())
    )
    result["percent_delayed"] = (result["delayed_flights"]/result["total_flights"]*100)
    result = result.reset_index()
    return render_template("index.html",
                           title="Airport Summary",
                           table=result.to_html(classes="table table-bordered table-striped", index=False))

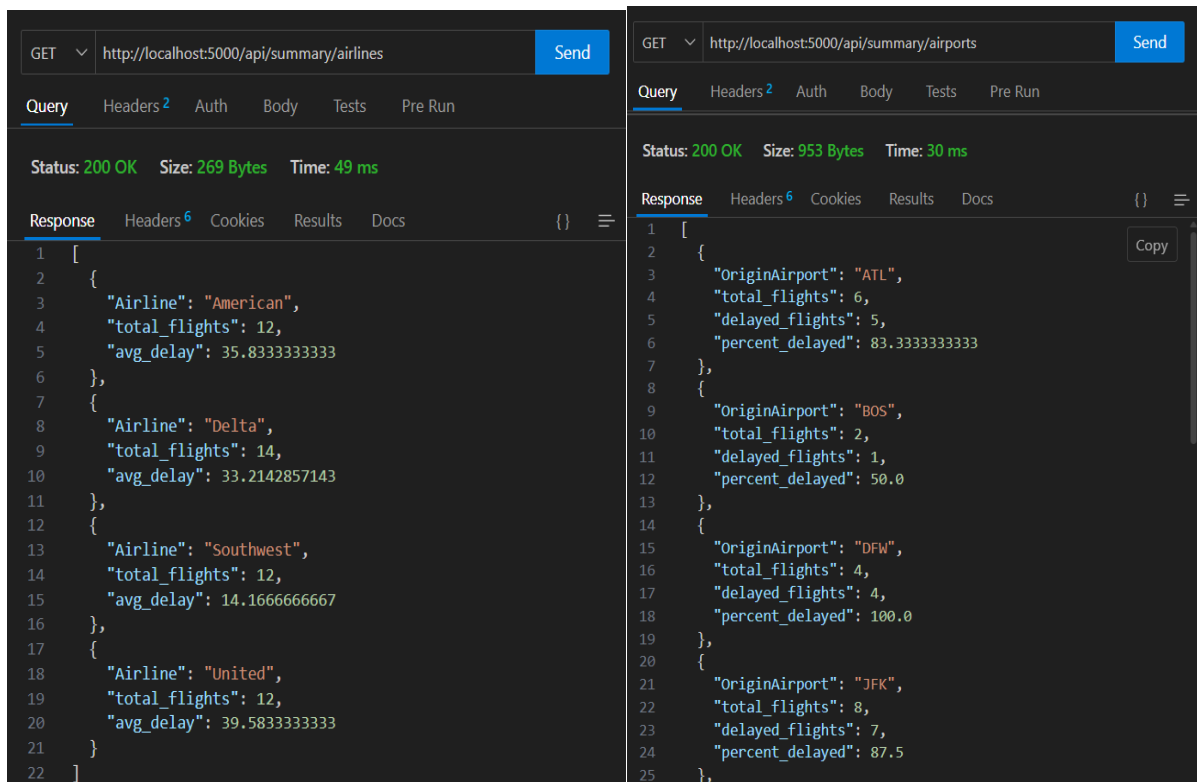
@app.route("/summary/routes")
def summary_routes():
    temp = df.copy()
    temp["Route"] = temp["OriginAirport"] + "-" + temp["DestinationAirport"]
    result = temp.groupby("Route").agg(
        avg_delay=("DepartureDelay", "mean"),
        total_flights=("FlightID", "count")
    ).reset_index()
    return render_template("index.html",
                           title="Route Summary",
                           table=result.to_html(classes="table table-bordered table-striped", index=False))
```

The Python code extends the Flask application with new routes to generate data summaries. Each route performs a different aggregation on the flight data, grouping by Airline, OriginAirport, and Route. It calculates metrics such as average delay, total flights, and the percentage of delayed flights. The results are then formatted into an HTML table and rendered dynamically on the web page.

Thunderclient

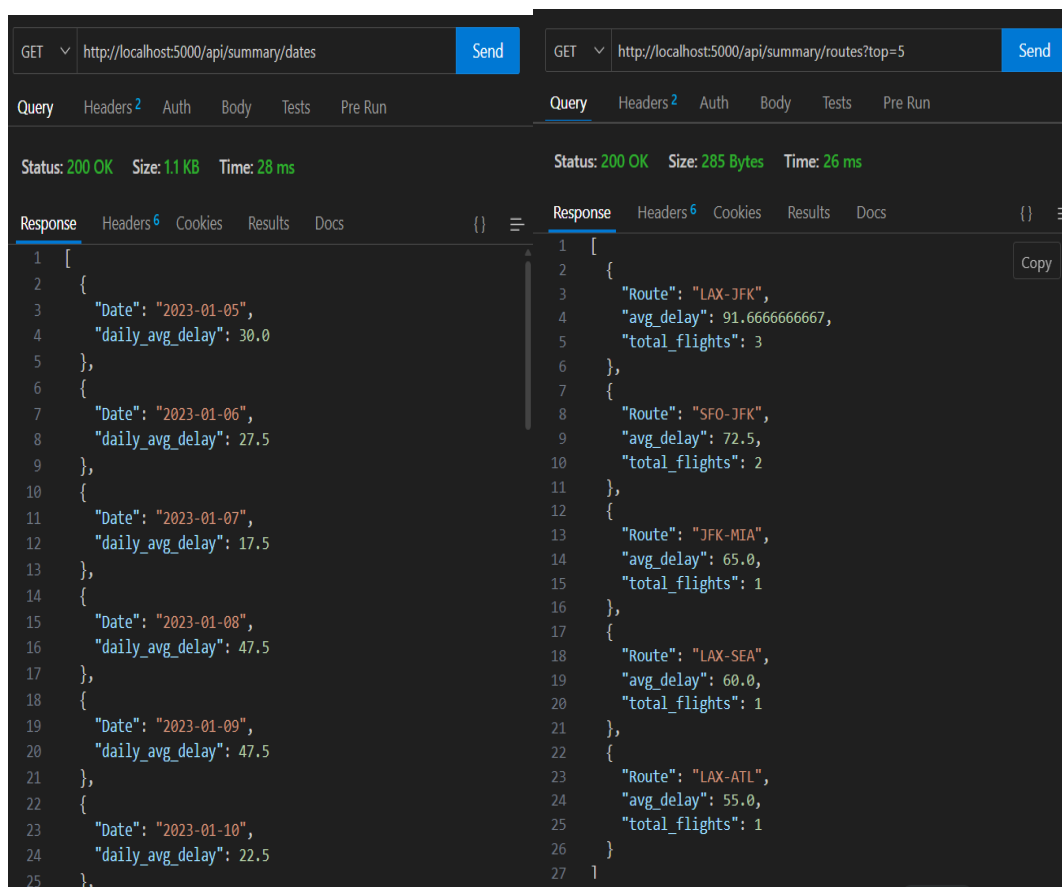


The screenshot shows the successful result of a GET request to a health check endpoint at `http://localhost:5000/api/health`. The response indicates a Status: 200 OK, confirming that the API is running correctly. The JSON response body `{"status": "ok"}` provides a clear and concise message about the API's operational status.



The images show two successful GET requests to API endpoints, both returning a 200 OK status. The first request, to `/api/summary/airlines`, returns a JSON array summarizing flight data by airline, including average delay and total flights.

The second, to `/api/summary/airports`, provides a similar summary, detailing total flights and the percentage of delayed flights for each airport.



These display successful API calls to two different summary endpoints. The first image shows a GET request to `/api/summary/dates`, returning a daily summary of average flight delays. The second, a request to `/api/summary/routes?top=5`, returns the top 5 routes with the highest average delays and their corresponding flight counts.

Output

```
C:\Users\Chandana K T\Downloads\project\project>python analysis.py

--- First 5 Rows ---
  FlightID  Airline OriginAirport DestinationAirport  Date  DepartureDelay  ArrivalDelay  Dis
0    F001    Delta          JFK              LAX  2023-01-05          15          10  2475
1    F002   United          ORD              SFO  2023-01-05          45          50  1846
2    F003    Delta          ATL              MIA  2023-01-06          -5           0   595
3    F004  American          DFW              LAX  2023-01-06          60          55  1235
4    F005 Southwest          LAX              LAS  2023-01-07          10           5   236

--- Info ---
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 50 entries, 0 to 49
Data columns (total 8 columns):
#   Column                Non-Null Count  Dtype
---  ---                -
0   FlightID              50 non-null    object
1   Airline                50 non-null    object
2   OriginAirport          50 non-null    object
3   DestinationAirport     50 non-null    object
4   Date                  50 non-null    object
5   DepartureDelay         50 non-null    int64
6   ArrivalDelay           50 non-null    int64
7   Dis                    50 non-null    int64
dtypes: int64(3), object(5)
memory usage: 3.3+ KB
None

--- Describe ---
      DepartureDelay  ArrivalDelay      Dis
count      50.000000      50.000000      50.000000
mean        30.800000      27.900000     1244.900000
std         26.483726      26.109854      832.605653
min        -10.000000      -5.000000     187.000000
25%         11.250000      10.000000     370.000000
50%         25.000000      22.500000     1216.000000
75%         45.000000      45.000000     1921.000000
max        100.000000      95.000000     2586.000000
```

The output, demonstrates the initial data analysis of a flights dataset using the pandas library. It shows the first five rows of the DataFrame, a summary of data types and non-null counts, and descriptive statistics for numerical columns like DepartureDelay, ArrivalDelay, and Dis.

```
Flights delayed > 30 mins:
  FlightID  Airline OriginAirport DestinationAirport  Date  DepartureDelay  ArrivalDelay  Dis
1    F002   United          ORD              SFO  2023-01-05          45          50  1846
3    F004  American          DFW              LAX  2023-01-06          60          55  1235
6    F007   United          SFO              JFK  2023-01-08          75          80  2586
9    F010    Delta          LAX              JFK  2023-01-09          90          85  2475
12   F013 Southwest          LAX              SFO  2023-01-11          40          35   337

Flights from JFK:
  FlightID  Airline OriginAirport DestinationAirport  Date  DepartureDelay  ArrivalDelay  Dis
0    F001    Delta          JFK              LAX  2023-01-05          15          10  2475
5    F006    Delta          JFK              ORD  2023-01-07          25          20   740
13   F014    Delta          JFK              BOS  2023-01-11         -10           -5   187
23   F024  American          JFK              MIA  2023-01-16          65          60  1090
26   F027   United          JFK              SFO  2023-01-18          20          25  2586

Flights by Delta:
  FlightID  Airline OriginAirport DestinationAirport  Date  DepartureDelay  ArrivalDelay  Dis
0    F001    Delta          JFK              LAX  2023-01-05          15          10  2475
2    F003    Delta          ATL              MIA  2023-01-06          -5           0   595
5    F006    Delta          JFK              ORD  2023-01-07          25          20   740
9    F010    Delta          LAX              JFK  2023-01-09          90          85  2475
13   F014    Delta          JFK              BOS  2023-01-11         -10           -5   187

--- Summaries exported successfully! ---
```

The output, shows the results of various data analysis and filtering operations on a flights dataset. It presents key summaries like the average delay per airline and the percentage of delayed flights per airport, as well as a list of flights that are delayed by more than 30 minutes, originate from JFK, or are operated by Delta.

Web-page

✈ Flight Analysis Dashboard

- 🕒 Flights Delayed > 30 mins
- 📍 Flights from JFK
- ✈ Flights by Delta
- 📊 Airline Summary
- ✈ Airport Summary
- 📅 Daily Avg Delay
- 📈 Top 5 Routes by Delay

Click a button to display data

Dashboard

✈ Flight Analysis Dashboard

- 🕒 Flights Delayed > 30 mins
- 📍 Flights from JFK
- ✈ Flights by Delta
- 📊 Airline Summary
- ✈ Airport Summary
- 📅 Daily Avg Delay
- 📈 Top 5 Routes by Delay

FlightID	Airline	OriginAirport	DestinationAirport	Date	DepartureDelay	ArrivalDelay	Distance
F002	United	ORD	SFO	2023-01-05	45	50	1846
F004	American	DFW	LAX	2023-01-06	60	55	1235
F007	United	SFO	JFK	2023-01-08	75	80	2586
F010	Delta	LAX	JFK	2023-01-09	90	85	2475
F013	Southwest	LAX	SFO	2023-01-11	40	35	337
F015	United	LAX	SEA	2023-01-12	60	55	954
F018	Delta	ATL	LAX	2023-01-13	35	30	1946
F020	American	LAX	DFW	2023-01-14	50	45	1235
F024	American	JFK	MIA	2023-01-16	65	60	1090
F025	Southwest	SFO	LAX	2023-01-17	35	30	337
F026	Delta	LAX	ATL	2023-01-17	55	50	1946
F030	Delta	JFK	LAX	2023-01-19	80	75	2475
F031	United	ORD	SEA	2023-01-20	40	35	1721
F035	United	SFO	JFK	2023-01-22	70	65	2586
F039	United	ORD	LAX	2023-01-24	50	45	1744

Results of Flights delayed greater than 30minutes

✈ Flight Analysis Dashboard

- 🕒 Flights Delayed > 30 mins
- 📍 Flights from JFK
- ✈ Flights by Delta
- 📄 Airline Summary
- ✈ Airport Summary
- 📅 Daily Avg Delay
- 📊 Top 5 Routes by Delay

FlightID	Airline	OriginAirport	DestinationAirport	Date	DepartureDelay	ArrivalDelay	Distance
F001	Delta	JFK	LAX	2023-01-05	15	10	2475
F006	Delta	JFK	ORD	2023-01-07	25	20	740
F014	Delta	JFK	BOS	2023-01-11	-10	-5	187
F024	American	JFK	MIA	2023-01-16	65	60	1090
F027	United	JFK	SFO	2023-01-18	20	25	2586
F030	Delta	JFK	LAX	2023-01-19	80	75	2475
F036	American	JFK	ORD	2023-01-22	30	25	740
F046	Delta	JFK	DFW	2023-01-27	45	50	1391

Results of Flights that are from JFK

✈ Flight Analysis Dashboard

- 🕒 Flights Delayed > 30 mins
- 📍 Flights from JFK
- ✈ Flights by Delta
- 📄 Airline Summary
- ✈ Airport Summary
- 📅 Daily Avg Delay
- 📊 Top 5 Routes by Delay

FlightID	Airline	OriginAirport	DestinationAirport	Date	DepartureDelay	ArrivalDelay	Distance
F001	Delta	JFK	LAX	2023-01-05	15	10	2475
F003	Delta	ATL	MIA	2023-01-06	-5	0	595
F006	Delta	JFK	ORD	2023-01-07	25	20	740
F010	Delta	LAX	JFK	2023-01-09	90	85	2475
F014	Delta	JFK	BOS	2023-01-11	-10	-5	187
F018	Delta	ATL	LAX	2023-01-13	35	30	1946
F022	Delta	BOS	JFK	2023-01-15	5	0	187
F026	Delta	LAX	ATL	2023-01-17	55	50	1946
F030	Delta	JFK	LAX	2023-01-19	80	75	2475
F034	Delta	BOS	ATL	2023-01-21	0	-5	946
F038	Delta	ATL	MIA	2023-01-23	10	15	595
F042	Delta	LAX	JFK	2023-01-25	100	95	2475
F046	Delta	JFK	DFW	2023-01-27	45	50	1391
F050	Delta	ATL	LAX	2023-01-29	20	15	1946

Results of Flights that are by Delta

✈ Flight Analysis Dashboard

- 🕒 Flights Delayed > 30 mins
- 📍 Flights from JFK
- ✈ Flights by Delta
- 📄 Airline Summary
- ✈ Airport Summary
- 📅 Daily Avg Delay
- 📊 Top 5 Routes by Delay

Airline	total_flights	avg_delay
American	12	35.8333333333
Delta	14	33.2142857143
Southwest	12	14.1666666667
United	12	39.5833333333

Results of Airline summary

✈️ Flight Analysis Dashboard

- 🕒 Flights Delayed > 30 mins
- 📍 Flights from JFK
- ✈️ Flights by Delta
- 📋 Airline Summary
- ✈️ Airport Summary
- 📅 Daily Avg Delay
- 📊 Top 5 Routes by Delay

Date	daily_avg_delay
2023-01-05	30
2023-01-06	27.5
2023-01-07	17.5
2023-01-08	47.5
2023-01-09	47.5
2023-01-10	22.5
2023-01-11	15
2023-01-12	40
2023-01-13	17.5
2023-01-14	37.5
2023-01-15	10
2023-01-16	37.5
2023-01-17	45
2023-01-18	17.5
2023-01-19	42.5

Results of Airport summary

✈️ Flight Analysis Dashboard

- 🕒 Flights Delayed > 30 mins
- 📍 Flights from JFK
- ✈️ Flights by Delta
- 📋 Airline Summary
- ✈️ Airport Summary
- 📅 Daily Avg Delay
- 📊 Top 5 Routes by Delay

OriginAirport	total_flights	delayed_flights	percent_delayed
ATL	6	5	83.3333333333
BOS	2	1	50
DFW	4	4	100
JFK	8	7	87.5
LAS	4	3	75
LAX	11	11	100
MIA	3	3	100
ORD	5	5	100
PHX	2	1	50
SEA	1	1	100
SFO	4	4	100

Results of daily average delay of Flights

✈️ Flight Analysis Dashboard

- 🕒 Flights Delayed > 30 mins
- 📍 Flights from JFK
- ✈️ Flights by Delta
- 📋 Airline Summary
- ✈️ Airport Summary
- 📅 Daily Avg Delay
- 📊 Top 5 Routes by Delay

Route	avg_delay	total_flights
LAX-JFK	91.6666666667	3
SFO-JFK	72.5	2
JFK-MIA	65	1
LAX-SEA	60	1
LAX-ATL	55	1

Results of top 5 routes by delay

Testing & Results

Unit testing was performed on filtering functions and route responses. Integration testing ensured templates correctly displayed Pandas results. User acceptance testing confirmed that users could navigate, filter data, and view summaries successfully.

Conclusion

The Flight Delay Analysis & Visualization System successfully demonstrates how Python, Flask, and Pandas can be used to build an interactive data dashboard. The system provides meaningful insights into flight delays and airline performance.

Future Work

- Incorporating real-time flight data APIs.
- Adding predictive analytics for delay forecasting.
- Enhancing visualization using advanced charting libraries (Plotly, D3.js).
- Deploying on cloud platforms for wider accessibility.